PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Complete Report with Flowchart Diagrams, Pseudocode, Python Programs and Sample Outputs

Name: Nallajeru Praveen kumar

Roll Number: 26001406

Branch/Section: CSD-1

Subject: Python Programming

Note: Ten problems have been selected from the supplied Python Activity-Based Learning case-study list. Sample inputs and outputs are illustrative. The flowchart is presented in diagram form for each problem.

Selected Case Studies

1. 1. Employee Payroll and Deduction Analysis (Original case-study number: 4)
2. 2. Retail Billing with Multiple Discount Rules (Original case-study number: 7)
3. 3. Examination Result and Grace-Mark Validation (Original case-study number: 11)
4. 4. ATM Transaction Sequence Simulation (Original case-study number: 13)
5. 5. Smart Parking Fee Management (Original case-study number: 15)
6. 6. Fuel Consumption and Route Comparison (Original case-study number: 18)
7. 7. Password Policy Validation Engine (Original case-study number: 29)
8. 8. Student Feedback Sentiment Scoring (Original case-study number: 34)
9. 9. Student Marks Matrix Analysis (Original case-study number: 42)
10. 10. Food Waste Monitoring System (Original case-study number: 53)

Shape key for flowcharts: Oval = Start/Stop; Parallelogram = Input; Rectangle = Processing/Output; Diamond = Decision.

Case Study 1: Employee Payroll and Deduction Analysis

Selected from original case-study number 4.

1. Problem Statement

Develop a Python program to calculate an employee's gross salary from basic pay, allowances, overtime and incentives. Apply deductions and display a formatted salary statement.

2. Objectives

- Calculate total allowances and gross salary.
- Calculate deductions and net salary.
- Display a clear salary statement.

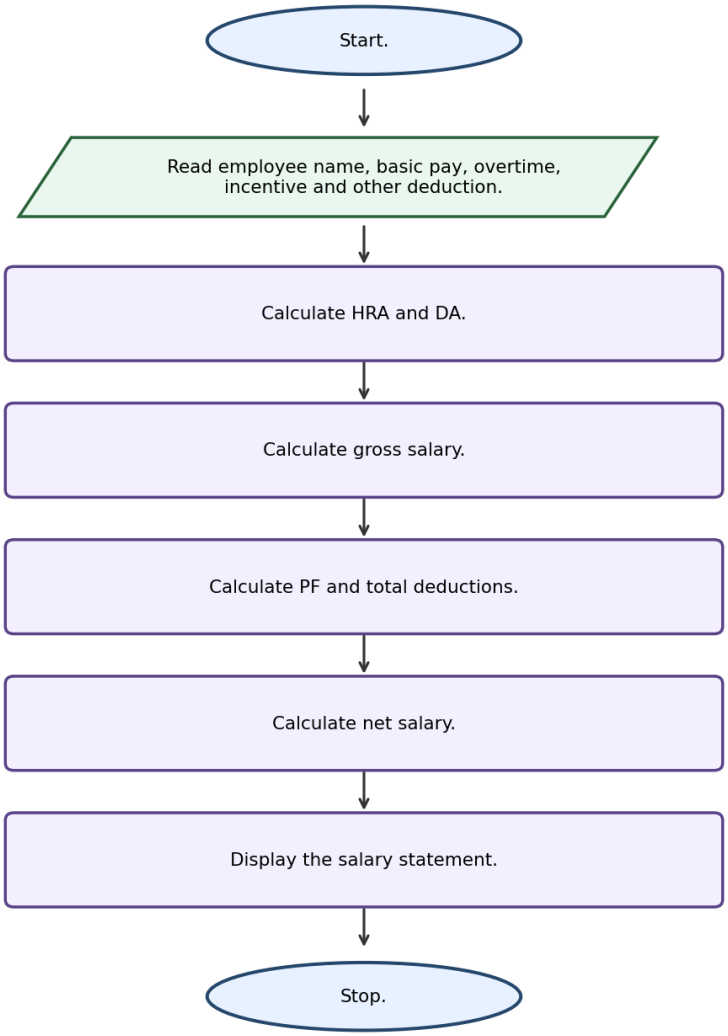
3. Assumptions

- All amounts are entered in Indian rupees.
- HRA is 20% and DA is 10% of basic pay.
- Overtime and incentives are entered as amounts.
- Provident fund deduction is 12% of basic pay and other deduction is entered by the user.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

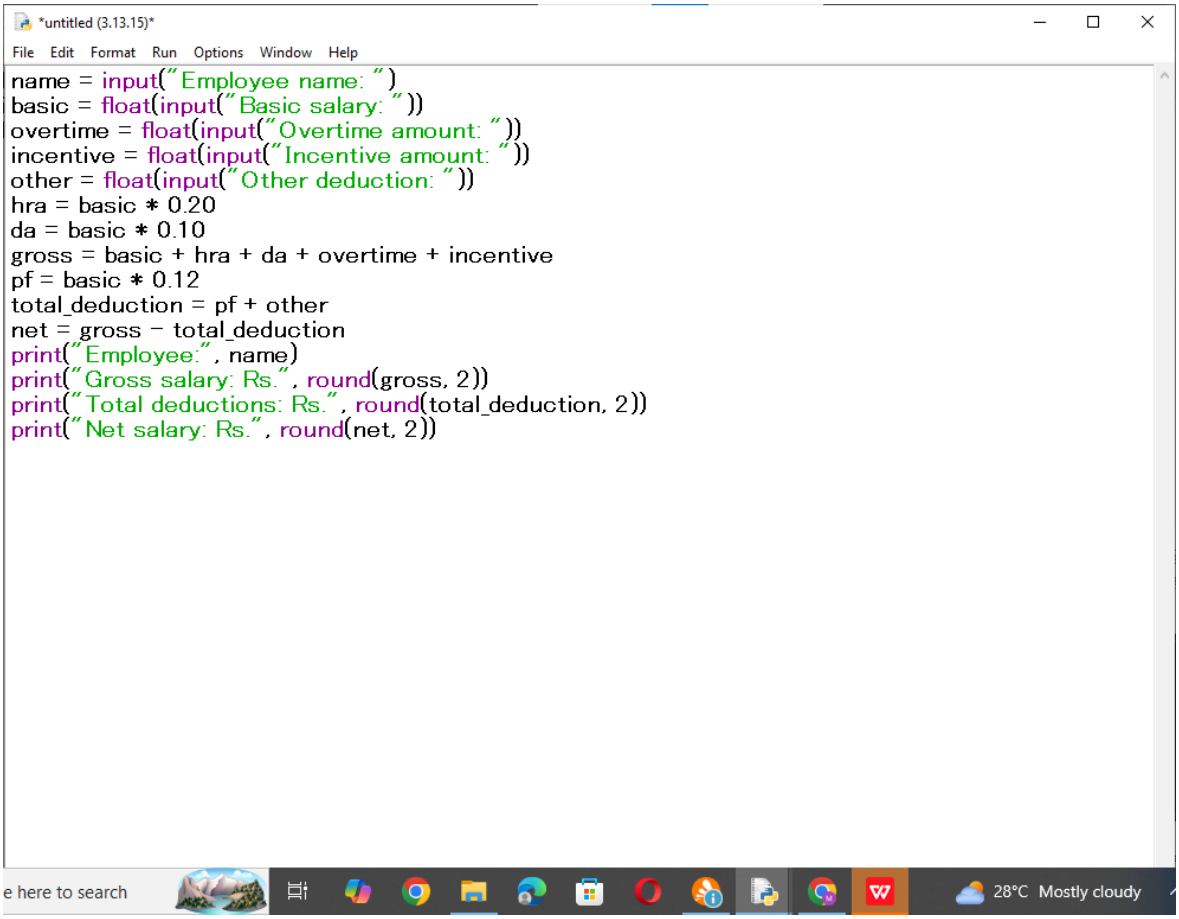
11. Start.
12. Read employee name, basic pay, overtime, incentive and other deduction.
13. Calculate HRA and DA.
14. Calculate gross salary.
15. Calculate PF and total deductions.
16. Calculate net salary.
17. Display the salary statement.
18. Stop.

5. Pseudocode

```
BEGIN
  INPUT name, basic, overtime, incentive, other_deduction
```

```
  hra = basic * 0.20
  da = basic * 0.10
  gross = basic + hra + da + overtime + incentive
  pf = basic * 0.12
  deductions = pf + other_deduction
  net = gross - deductions
  DISPLAY salary statement
END
```

6. Python Source Code



7.

7. Test Cases

Input / Test	Expected result
Ravi; basic=20000, overtime=1000, incentive=500, other=300	Gross=Rs. 26500.00; deductions=Rs. 2700.00; net=Rs. 23800.00

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program calculates gross salary, deductions and net salary using basic arithmetic operations and displays an employee salary statement.

Case Study 2: Retail Billing with Multiple Discount Rules

Selected from original case-study number 7.

1. Problem Statement

Create a retail billing program that accepts product prices and quantities, calculates the subtotal, applies a discount when the bill reaches a given threshold, adds tax and prints the final invoice.

2. Objectives

- Calculate item-wise and total cost.
- Apply a threshold-based discount.
- Calculate tax and final amount.

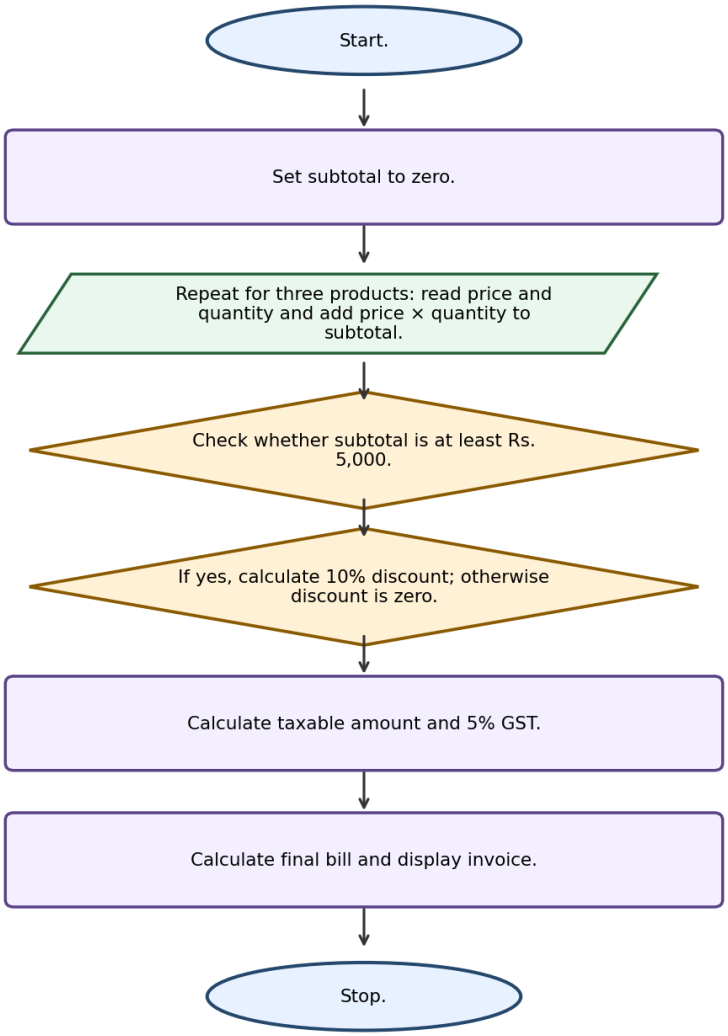
3. Assumptions

- The program accepts three products.
- A 10% discount is applied when subtotal is Rs. 5,000 or more.
- GST is calculated at 5% after discount.
- All prices and quantities are positive.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

19. Start.
20. Set subtotal to zero.
21. Repeat for three products: read price and quantity and add price × quantity to subtotal.
22. Check whether subtotal is at least Rs. 5,000.
23. If yes, calculate 10% discount; otherwise discount is zero.
24. Calculate taxable amount and 5% GST.
25. Calculate final bill and display invoice.
26. Stop.

5. Pseudocode

```
BEGIN
  subtotal = 0
```

```
  REPEAT 3 TIMES
    INPUT price, quantity
    subtotal = subtotal + price * quantity
  END REPEAT
  IF subtotal >= 5000 THEN
    discount = subtotal * 0.10
  ELSE
    discount = 0
  END IF
  taxable = subtotal - discount
  tax = taxable * 0.05
  total = taxable + tax
  DISPLAY subtotal, discount, tax, total
END
```

6. Python Source Code



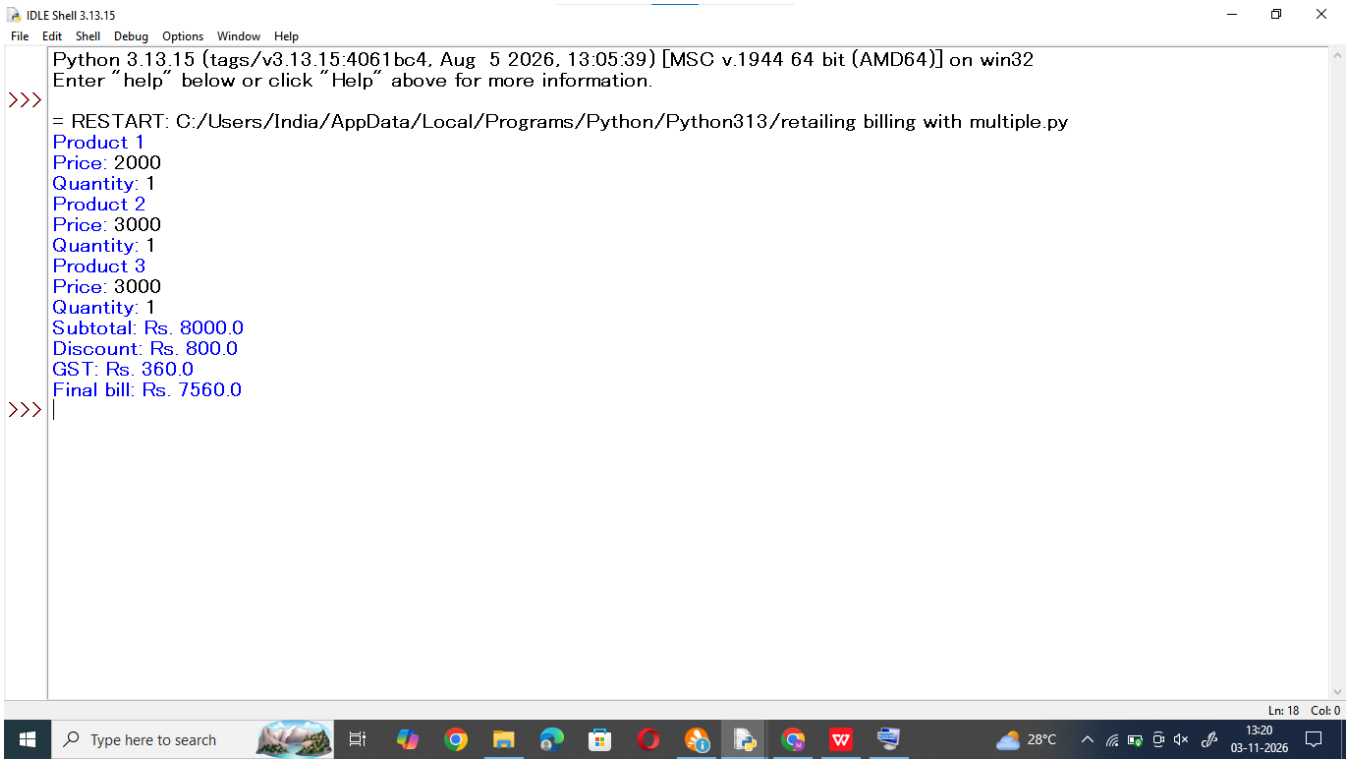
7.

7. Test Cases

Input / Test	Expected result
Prices/quantities: 2000×1, 1500×1, 2000×1	Subtotal=5500; discount=550; GST=247.50; final=5197.50

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program uses a loop, conditional statement and arithmetic operators to calculate a retail bill with discount and GST.

Case Study 3: Examination Result and Grace-Mark Validation

Selected from original case-study number 11.

1. Problem Statement

Write a program to process marks in multiple subjects, apply a limited grace-mark rule and display the final result without allowing marks to exceed the maximum.

2. Objectives

- Accept marks in subjects.
- Apply a grace mark only when the student is within the permitted range.
- Display subject results and overall result.

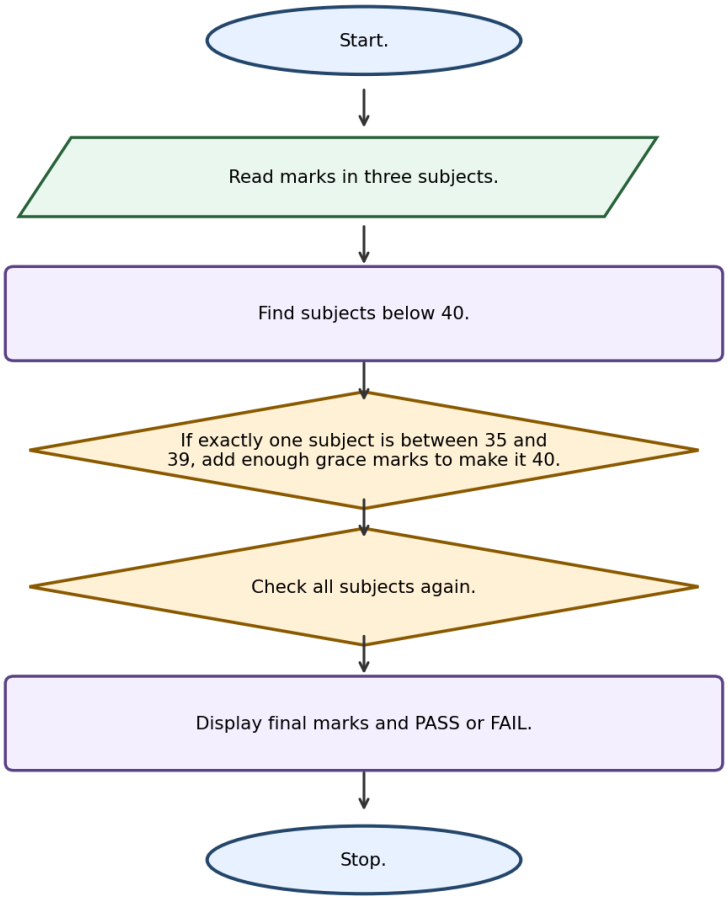
3. Assumptions

- There are three subjects, each out of 100.
- Pass mark is 40 in each subject.
- A student short by at most 5 marks in exactly one subject receives the required grace marks.
- Final marks cannot exceed 100.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

27. Start.
28. Read marks in three subjects.
29. Find subjects below 40.
30. If exactly one subject is between 35 and 39, add enough grace marks to make it 40.
31. Check all subjects again.
32. Display final marks and PASS or FAIL.
33. Stop.

5. Pseudocode

```
BEGIN
  INPUT marks for 3 subjects
  failed = subjects with marks < 40
```

```
  IF number of failed subjects = 1 AND failed mark >= 35 THEN
    add grace marks to make it 40
  END IF
  IF every mark >= 40 THEN
    result = PASS
  ELSE
    result = FAIL
  END IF
  DISPLAY marks and result
END
```

6. Python Source Code

```
marks = [ ]
for i in range(1, 4):
    mark = int(input("Enter marks for subject " + str(i) + ": "))
    marks.append(mark)

failed = [i for i in range(3) if marks[i] < 40]

if len(failed) == 1 and marks[failed[0]] >= 35:
    index = failed[0]
    grace = 40 - marks[index]
    marks[index] = min(100, marks[index] + grace)
    print("Grace marks awarded:", grace)

print("Final marks:", marks)
if all(mark >= 40 for mark in marks):
    print("Result: PASS")
else:
    print("Result: FAIL")
```

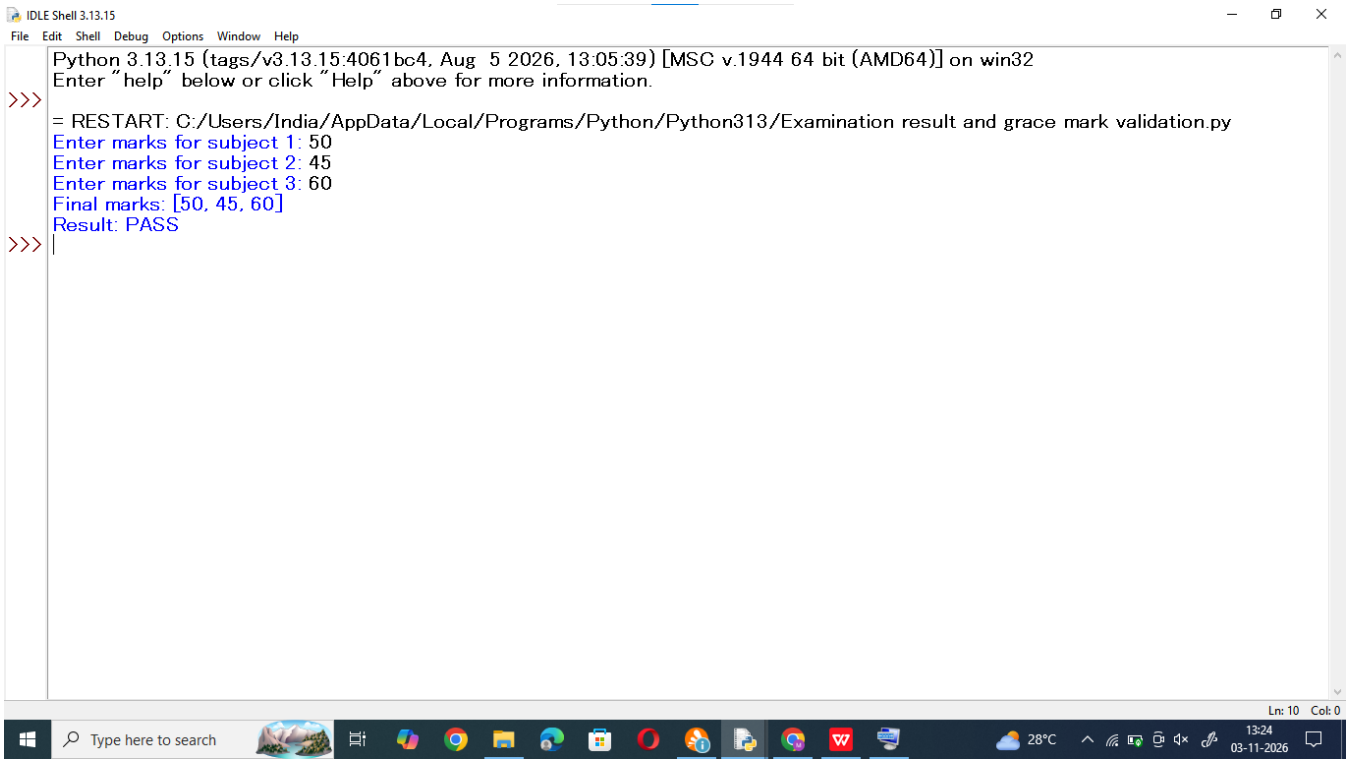
7.

7. Test Cases

Input / Test	Expected result
Marks: 45, 38, 67	Grace=2; final marks=[45, 40, 67]; PASS
Marks: 32, 38, 67	Two failed subjects; FAIL

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program uses lists and conditional statements to apply a simple grace-mark policy and decide the examination result.

Case Study 4: ATM Transaction Sequence Simulation

Selected from original case-study number 13.

1. Problem Statement

Develop a simple ATM simulation that processes deposits and withdrawals, maintains the account balance, rejects invalid withdrawals and displays the final balance.

2. Objectives

- Perform deposit and withdrawal operations.
- Prevent withdrawals greater than the available balance.
- Show the final account balance.

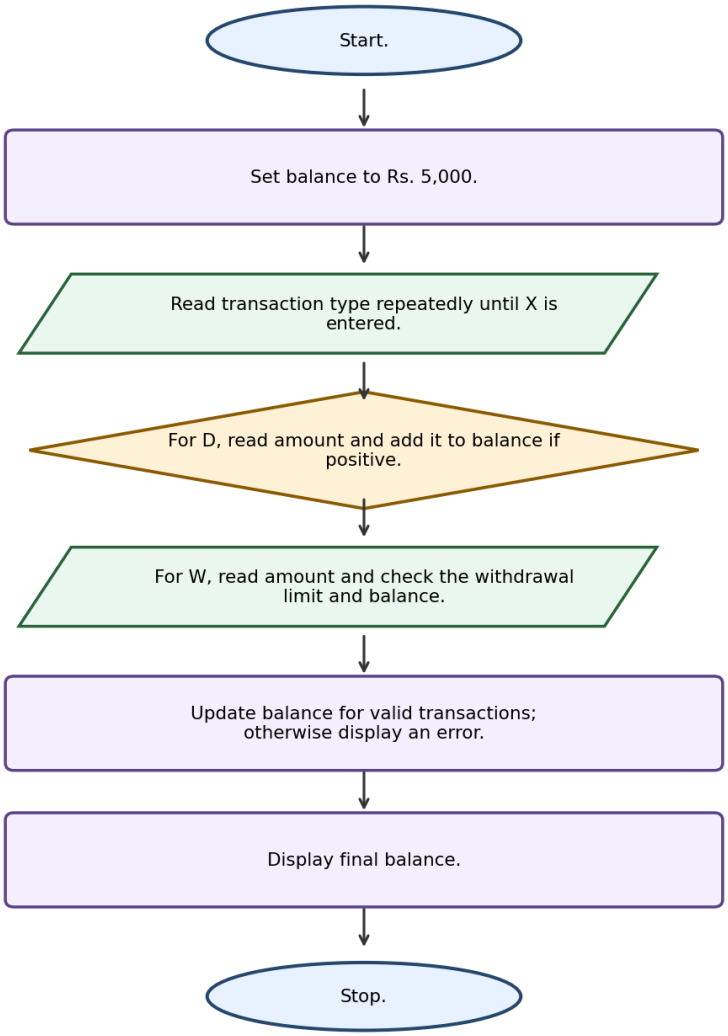
3. Assumptions

- Opening balance is Rs. 5,000.
- The user enters D for deposit or W for withdrawal.
- Each withdrawal is limited to Rs. 10,000 and cannot exceed the balance.
- Enter X to finish transactions.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

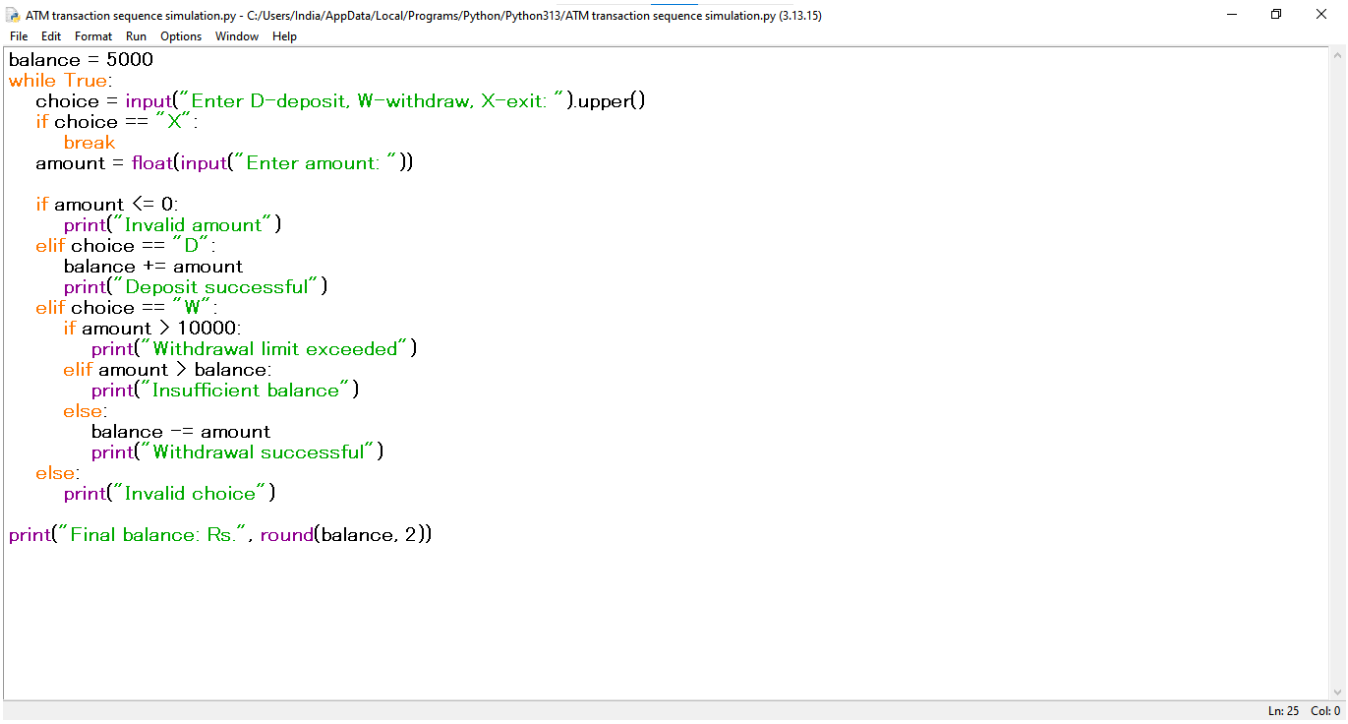
34. Start.
35. Set balance to Rs. 5,000.
36. Read transaction type repeatedly until X is entered.
37. For D, read amount and add it to balance if positive.
38. For W, read amount and check the withdrawal limit and balance.
39. Update balance for valid transactions; otherwise display an error.
40. Display final balance.
41. Stop.

5. Pseudocode

```
BEGIN
  balance = 5000
```

```
  INPUT transaction_type
  WHILE transaction_type != X
    IF type = D THEN
      INPUT amount
      IF amount > 0: balance = balance + amount
    ELSE IF type = W THEN
      INPUT amount
      IF amount > 0 AND amount <= 10000 AND amount <= balance
        balance = balance - amount
      ELSE DISPLAY invalid transaction
    END IF
    INPUT next transaction_type
  END WHILE
  DISPLAY balance
END
```

6. Python Source Code



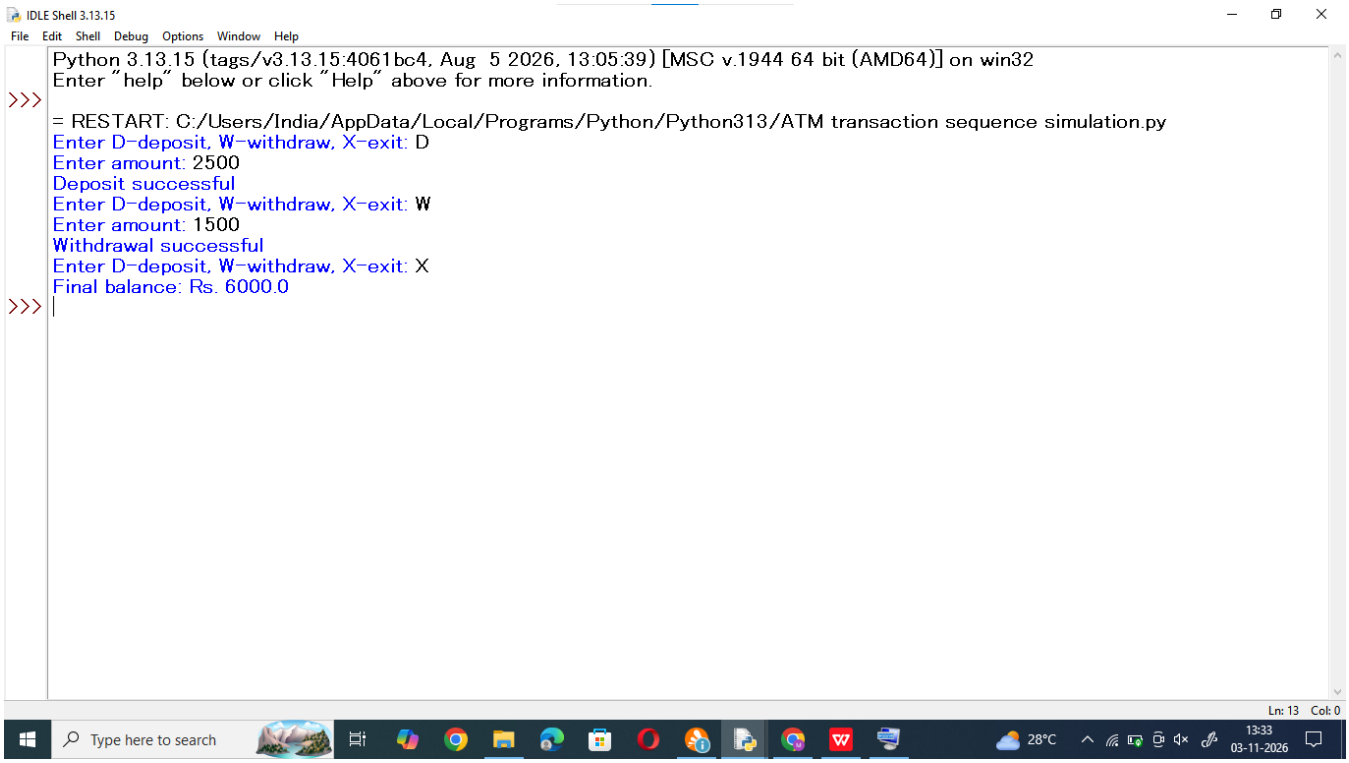
7.

7. Test Cases

Input / Test	Expected result
Deposit 2000; withdraw 1500; exit	Final balance=Rs. 5500.00
Withdraw 7000 from opening balance	Insufficient balance; balance remains Rs. 5000

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The ATM simulation demonstrates loops, selection statements and balance updates while checking transaction limits.

Case Study 5: Smart Parking Fee Management

Selected from original case-study number 15.

1. Problem Statement

Create a parking fee calculator that reads the number of hours a vehicle is parked and calculates the parking charge based on a simple hourly rate.

2. Objectives

- Accept parking duration and vehicle category.
- Calculate parking charges.
- Apply a maximum daily charge.

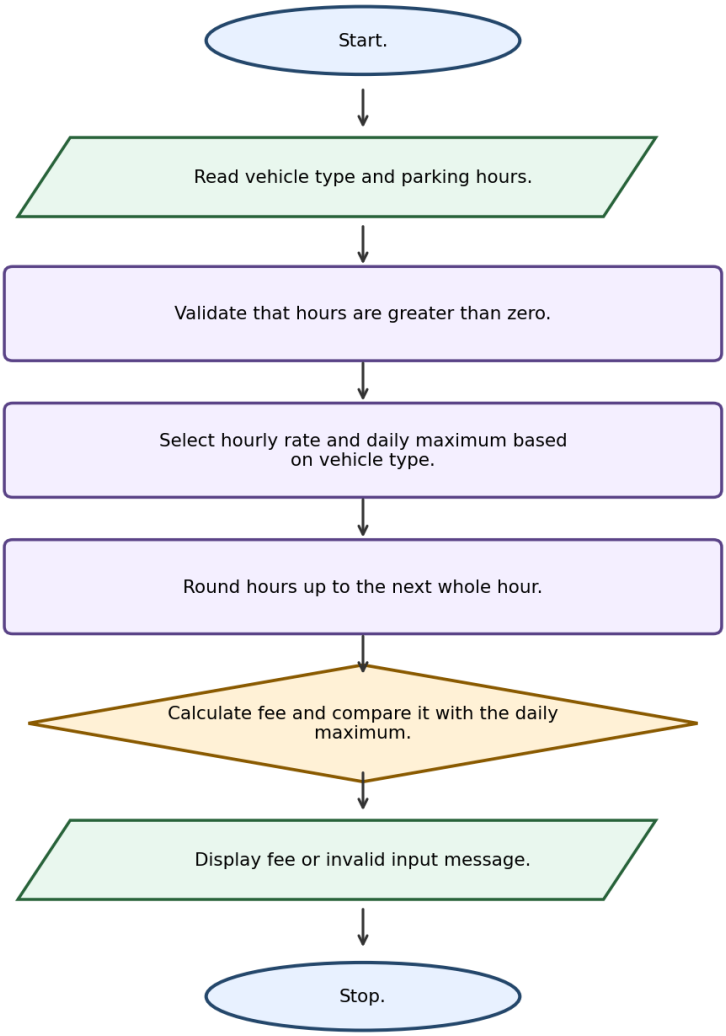
3. Assumptions

- Car parking costs Rs. 30 per hour.
- Bike parking costs Rs. 15 per hour.
- Any part of an hour is charged as a full hour.
- Maximum charge is Rs. 200 for a car and Rs. 100 for a bike per day.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

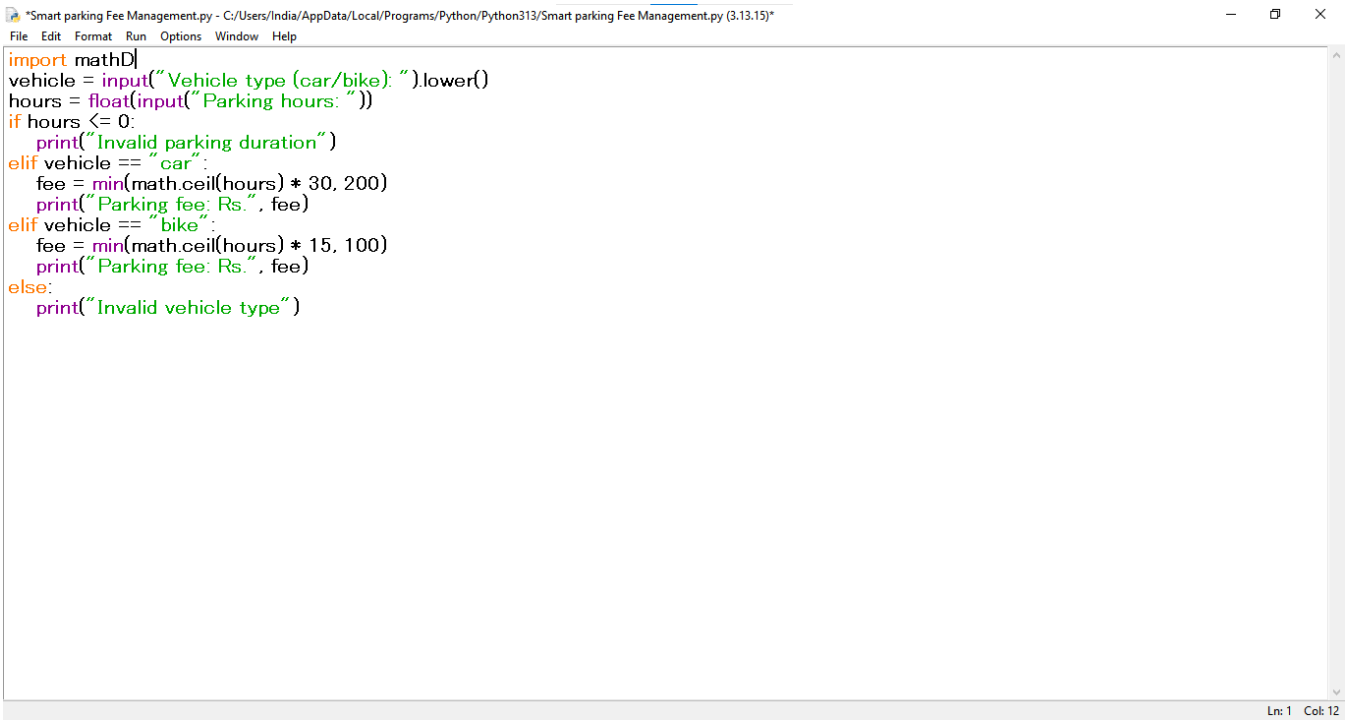
42. Start.
43. Read vehicle type and parking hours.
44. Validate that hours are greater than zero.
45. Select hourly rate and daily maximum based on vehicle type.
46. Round hours up to the next whole hour.
47. Calculate fee and compare it with the daily maximum.
48. Display fee or invalid input message.
49. Stop.

5. Pseudocode

```
BEGIN
  INPUT vehicle_type, hours
```

```
  IF hours <= 0 THEN DISPLAY invalid input
  ELSE
    IF vehicle_type = CAR: rate=30, limit=200
    ELSE IF vehicle_type = BIKE: rate=15, limit=100
    ELSE DISPLAY invalid vehicle
    charged_hours = ceiling(hours)
    fee = charged_hours * rate
    IF fee > limit: fee = limit
    DISPLAY fee
  END IF
END
```

6. Python Source Code

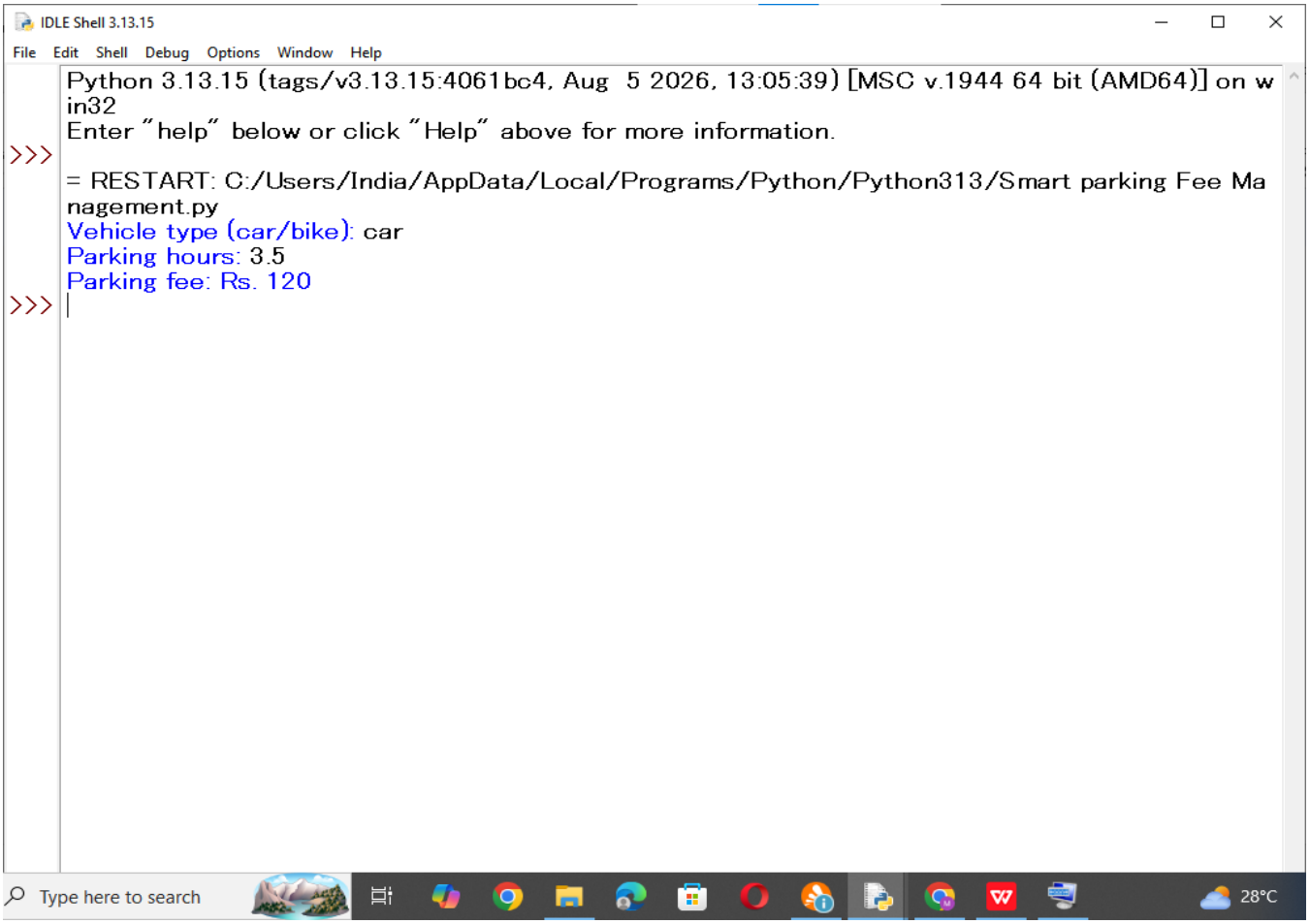


7. Test Cases

Input / Test	Expected result
car, 3.5 hours	4 charged hours × Rs.30 = Rs.120
bike, 9 hours	Rs.100 (daily maximum)

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program calculates parking fees using conditions, mathematical rounding and a maximum daily charge.

Case Study 6: Fuel Consumption and Route Comparison

Selected from original case-study number 18.

1. Problem Statement

Write a program to compare the fuel cost of different vehicles for a journey using distance, mileage and fuel price, and identify the lowest-cost vehicle.

2. Objectives

- Calculate fuel required for each vehicle.
- Calculate journey cost.
- Identify the vehicle with the lowest fuel cost.

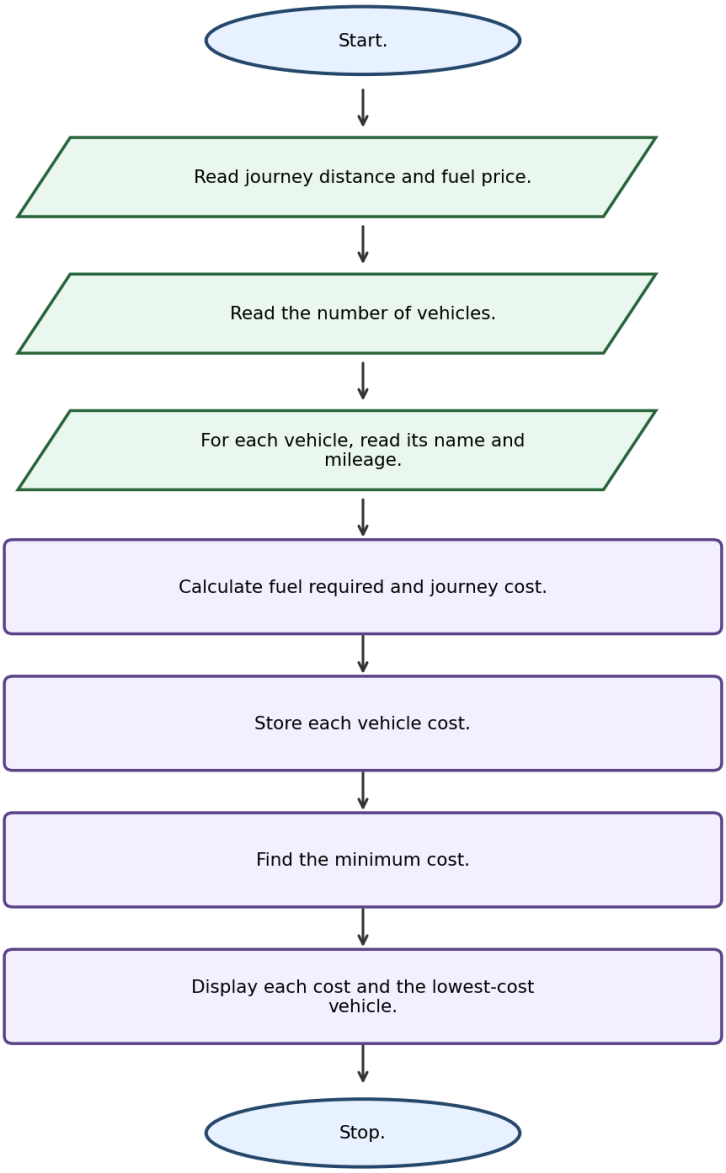
3. Assumptions

- The same journey distance and fuel price apply to all vehicles.
- Mileage is entered in kilometres per litre.
- Fuel cost is distance divided by mileage multiplied by fuel price.
- At least one vehicle is entered.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

50. Start.
51. Read journey distance and fuel price.
52. Read the number of vehicles.
53. For each vehicle, read its name and mileage.
54. Calculate fuel required and journey cost.
55. Store each vehicle cost.
56. Find the minimum cost.
57. Display each cost and the lowest-cost vehicle.
58. Stop.

5. Pseudocode

```
BEGIN
  INPUT distance, fuel_price, number_of_vehicles
  FOR each vehicle
    INPUT name, mileage
    fuel = distance / mileage
    cost = fuel * fuel_price
    STORE name and cost
  END FOR
  best = vehicle with minimum cost
  DISPLAY all costs and best vehicle
END
```

6. Python Source Code

Fuel consumption and Route comparison.py - C:/Users/India/AppData/Local/Programs/Python/Python313/Fuel consumption and Route comparison.py (3.13.15)

File Edit Format Run Options Window Help

```
distance = float(input("Journey distance (km): "))
fuel_price = float(input("Fuel price per litre: "))
n = int(input("Number of vehicles: "))

results = [ ]
for i in range(n):
    name = input("Vehicle name: ")
    mileage = float(input("Mileage (km/litre): "))
    if mileage > 0:
        fuel = distance / mileage
        cost = fuel * fuel_price
        results.append((name, cost))
        print(name, "fuel cost: Rs.", round(cost, 2))
    else:
        print("Mileage must be positive")

if results:
    best = min(results, key=lambda item: item[1])
    print("Lowest-cost vehicle:", best[0])
    print("Lowest cost: Rs.", round(best[1], 2))
```

Ln: 20 Col: 48

7.

7. Test Cases

Input / Test	Expected result
Distance=300 km, fuel=Rs.100, car mileage=15, bike mileage=30	Car cost=Rs.2000; bike cost=Rs.1000; bike is lowest cost

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program uses loops, tuples and the minimum function to compare vehicle fuel costs and select the economical option.

Case Study 7: Password Policy Validation Engine

Selected from original case-study number 29.

1. Problem Statement

Develop a Python program to check whether a password follows basic security rules such as minimum length, uppercase letter, lowercase letter, digit and special character.

2. Objectives

- Check the length of a password.
- Check required character types.
- Display whether the password is valid and list missing rules.

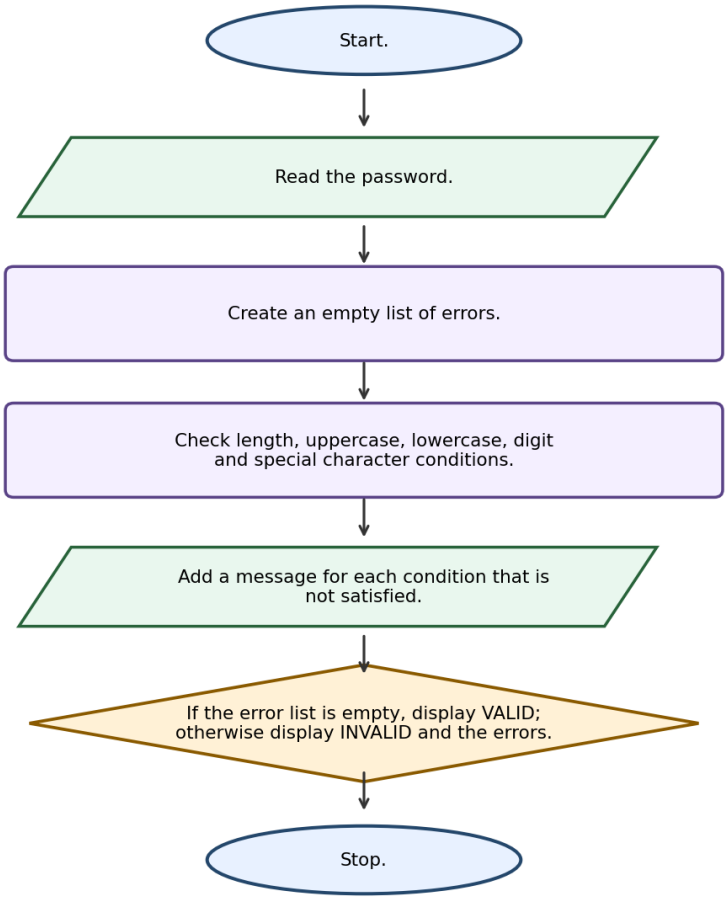
3. Assumptions

- A valid password must contain at least 8 characters.
- It must contain one uppercase letter, one lowercase letter, one digit and one special character.
- Whitespace is not accepted as a special character.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

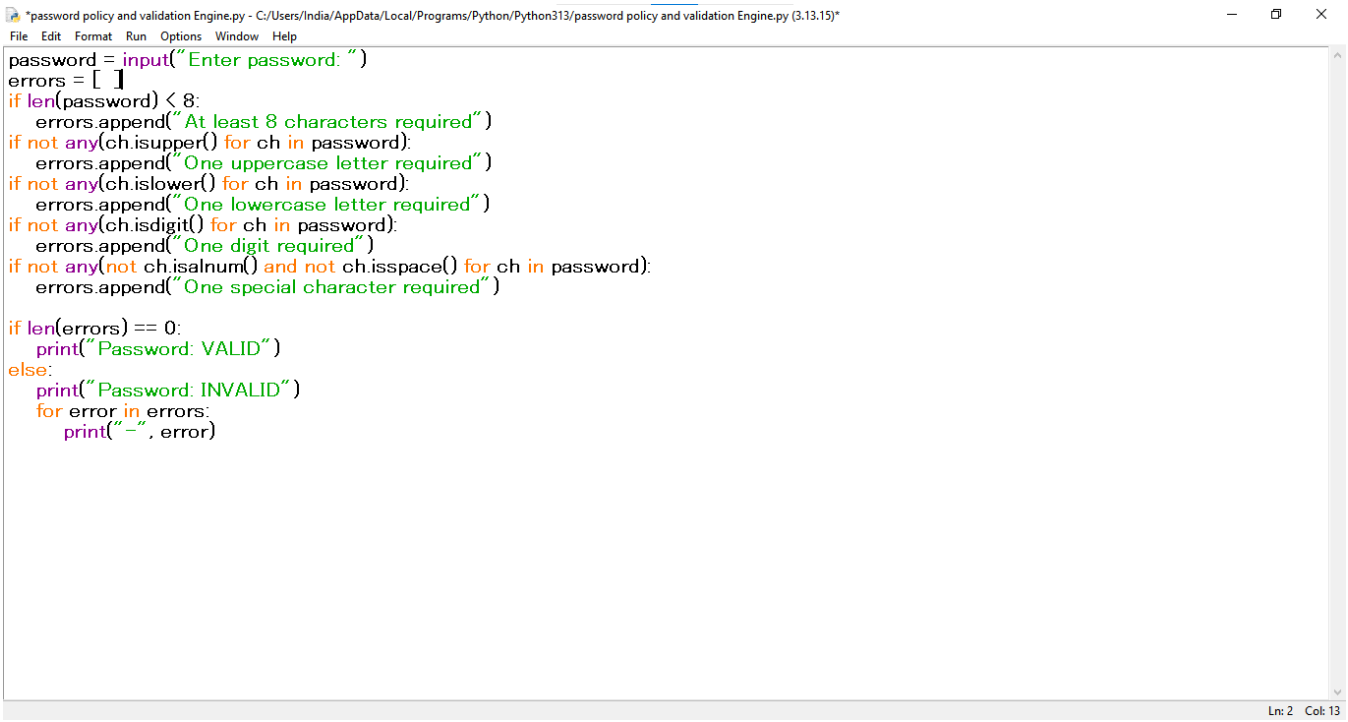
59. Start.
60. Read the password.
61. Create an empty list of errors.
62. Check length, uppercase, lowercase, digit and special character conditions.
63. Add a message for each condition that is not satisfied.
64. If the error list is empty, display VALID; otherwise display INVALID and the errors.
65. Stop.

5. Pseudocode

```
BEGIN
  INPUT password
  errors = empty list
```

```
  IF length < 8: add length error
  IF no uppercase: add uppercase error
  IF no lowercase: add lowercase error
  IF no digit: add digit error
  IF no special character: add special-character error
  IF errors is empty: DISPLAY VALID
  ELSE DISPLAY INVALID and errors
END
```

6. Python Source Code



7.

7. Test Cases

Input / Test	Expected result
Abc@1234	VALID
hello123	INVALID: uppercase and special character required

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program uses string methods, loops and conditional statements to validate a password against a set of rules.

Case Study 8: Student Feedback Sentiment Scoring

Selected from original case-study number 34.

1. Problem Statement

Create a program that analyses student feedback sentences using positive and negative keywords, calculates a score and classifies the feedback.

2. Objectives

- Read feedback text.
- Count positive and negative keywords.
- Calculate a sentiment score and classify feedback.

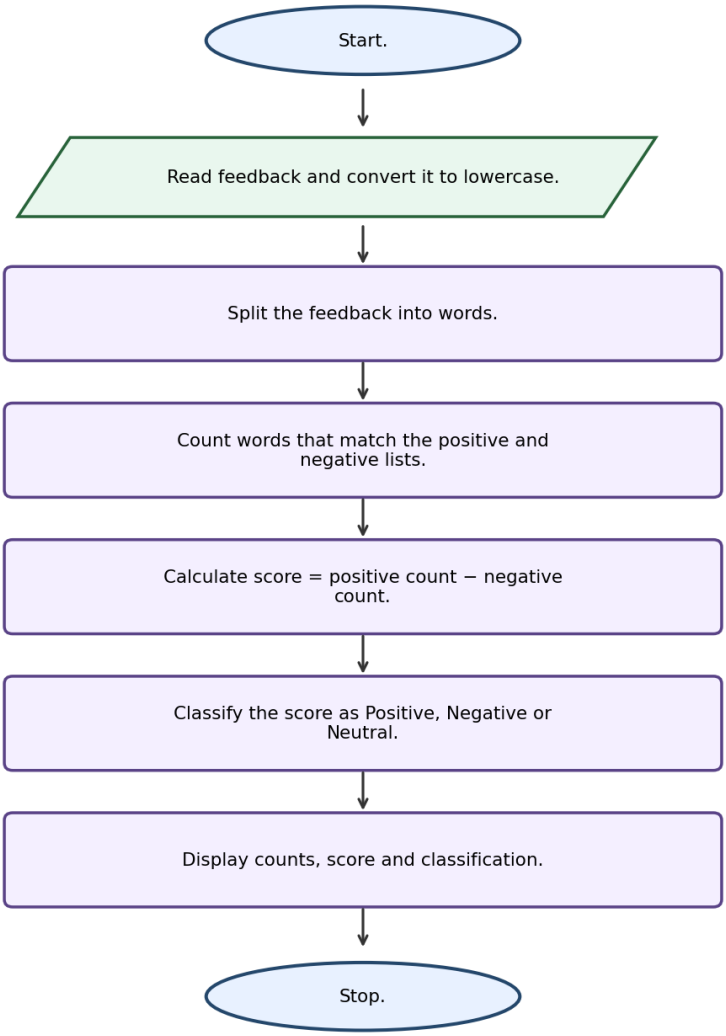
3. Assumptions

- The positive words are good, excellent, helpful and interesting.
- The negative words are bad, poor, difficult and boring.
- Each matching word adds one positive or negative point.
- A score above zero is Positive, below zero is Negative, and zero is Neutral.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

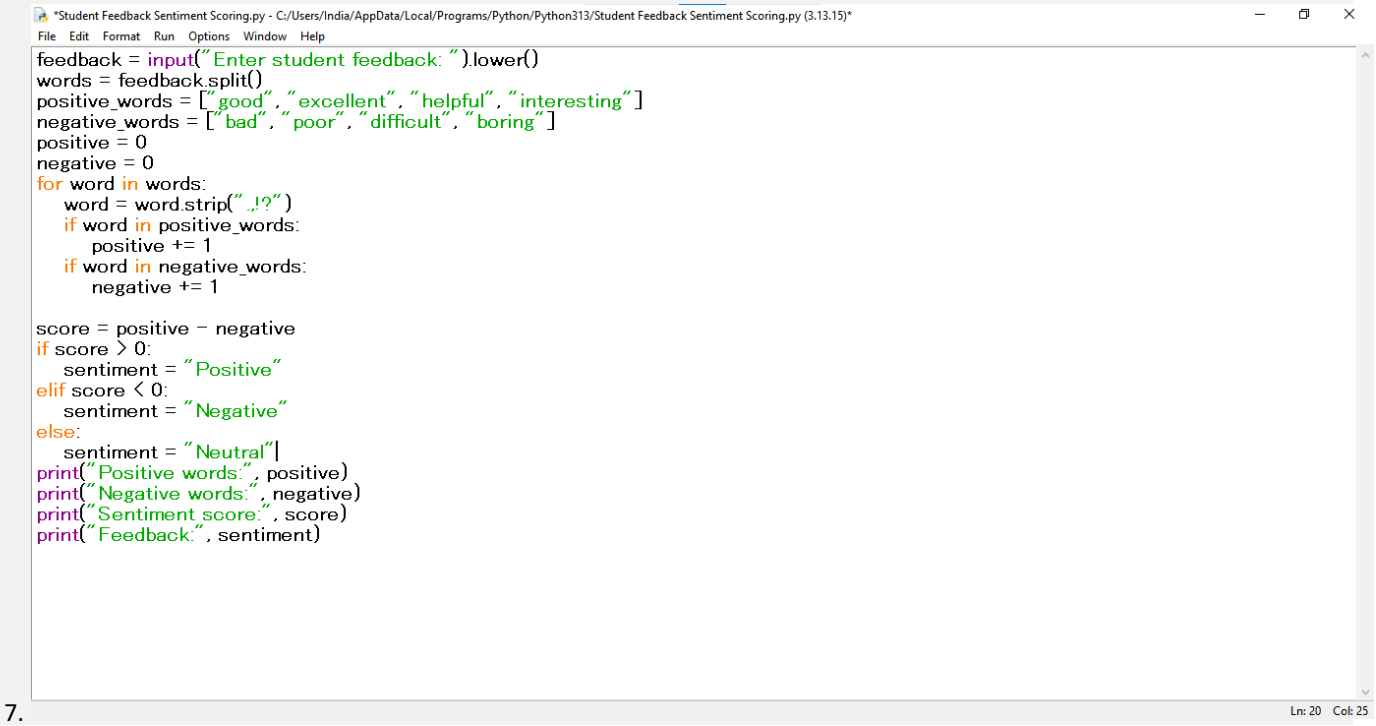
66. Start.
67. Read feedback and convert it to lowercase.
68. Split the feedback into words.
69. Count words that match the positive and negative lists.
70. Calculate score = positive count − negative count.
71. Classify the score as Positive, Negative or Neutral.
72. Display counts, score and classification.
73. Stop.

5. Pseudocode

```
BEGIN
  INPUT feedback
```

```
  words = lowercase feedback split into words
  positive_words = [good, excellent, helpful, interesting]
  negative_words = [bad, poor, difficult, boring]
  positive_count = number of matching positive words
  negative_count = number of matching negative words
  score = positive_count - negative_count
  IF score > 0: sentiment = Positive
  ELSE IF score < 0: sentiment = Negative
  ELSE: sentiment = Neutral
  DISPLAY counts, score, sentiment
END
```

6. Python Source Code



7.

7. Test Cases

Input / Test	Expected result
The class is good and helpful	Positive=2; negative=0; score=2; Positive
The topic is difficult and boring	Positive=0; negative=2; score=-2; Negative

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program performs simple keyword-based sentiment analysis using strings, lists, loops and if-elif-else statements.

Case Study 9: Student Marks Matrix Analysis

Selected from original case-study number 42.

1. Problem Statement

Write a program to analyse marks stored in a nested list. Calculate each student's total and average, calculate subject averages and identify the highest total.

2. Objectives

- Store marks in a nested list.
- Calculate student-wise totals and averages.
- Calculate subject-wise averages and identify the top student.

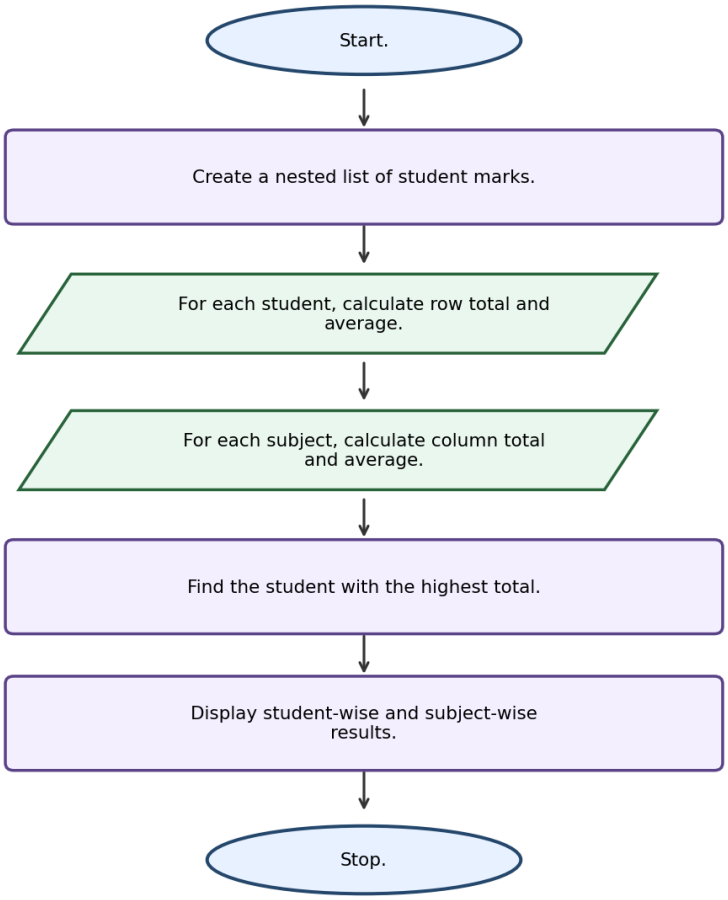
3. Assumptions

- Three students and three subjects are used in the sample.
- Every mark is out of 100.
- Each row represents one student and each column represents one subject.
- The subjects are Maths, Physics and Programming.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

74. Start.
75. Create a nested list of student marks.
76. For each student, calculate row total and average.
77. For each subject, calculate column total and average.
78. Find the student with the highest total.
79. Display student-wise and subject-wise results.
80. Stop.

5. Pseudocode

```
BEGIN
  marks = nested list of student marks
  FOR each student row
```

```
    total = sum(row)
    average = total / number of subjects
    DISPLAY total and average
  END FOR
  FOR each subject column
    subject_total = sum of that column
    subject_average = subject_total / number of students
    DISPLAY subject_average
  END FOR
  top_index = index of maximum row total
  DISPLAY top student
END
```

6. Python Source Code

7.

Student Marks Matrix Analysis.py - C:/Users/India/AppData/Local/Programs/Python/Python313/Student Marks Matrix Analysis.py (3.13.15)

File Edit Format Run Options Window Help

```
students = ["Anu", "Balu", "Charan"]
subjects = ["Maths", "Physics", "Programming"]
marks = [ [80, 75, 90],[65, 85, 70],[92, 88, 95] ]

totals = [ ]
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(students[i], "Total:", total, "Average:", round(average, 2))

print("¥nSubject averages:")
for j in range(len(subjects)):
    subject_total = 0
    for i in range(len(marks)):
        subject_total += marks[i][j]
    print(subjects[j], ":", round(subject_total / len(students), 2))

top = totals.index(max(totals))
print("Top student:", students[top], "with", totals[top], "marks")
```

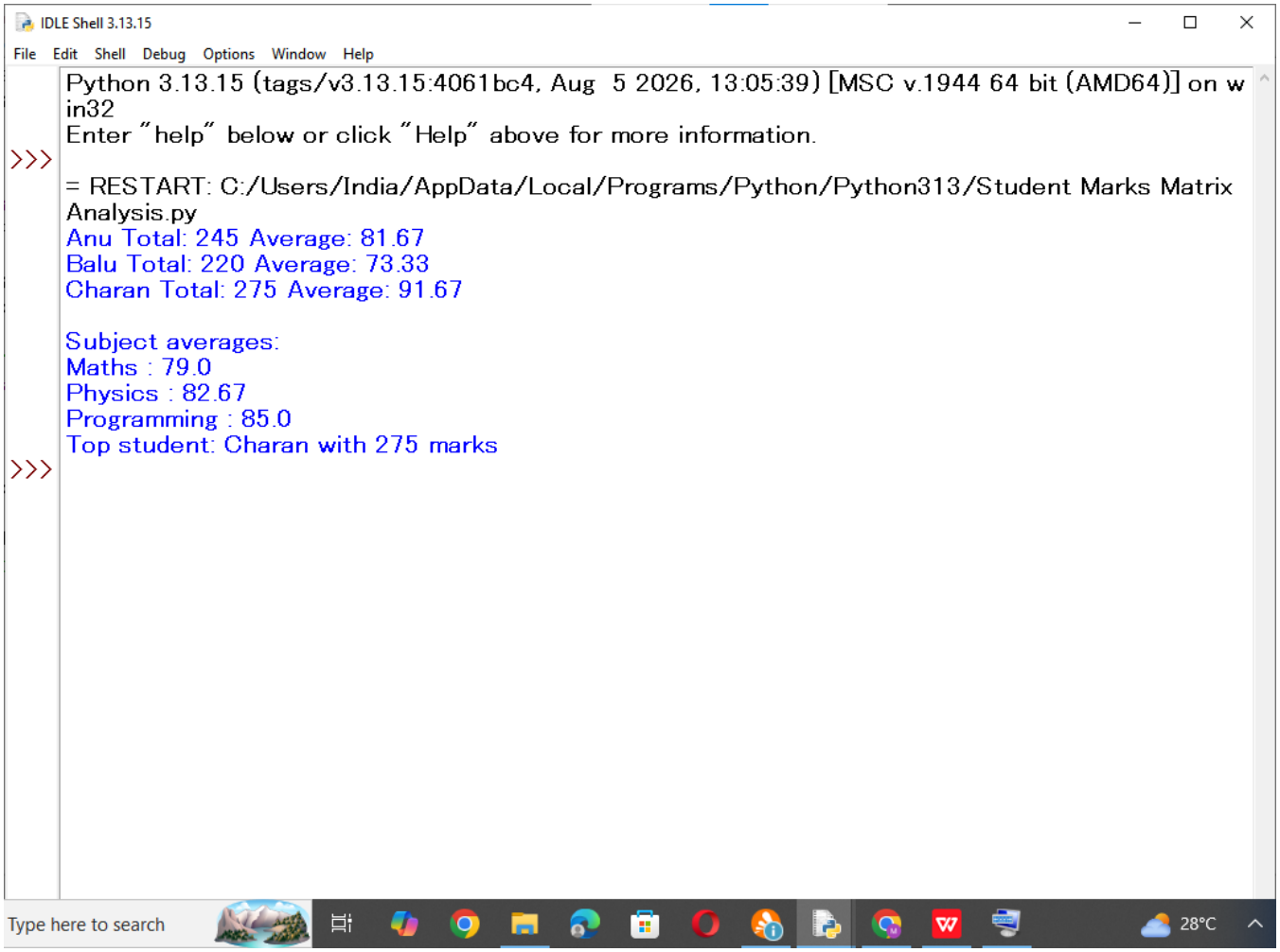
Ln: 21 Col: 0

7. Test Cases

Input / Test	Expected result
Rows: [80,75,90], [65,85,70], [92,88,95]	Totals=245,220,275; top student=Charan; subject averages=79.00,82.67,85.00

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program demonstrates nested lists and nested loops to calculate row-wise and column-wise statistics.

Case Study 10: Food Waste Monitoring System

Selected from original case-study number 53.

1. Problem Statement

Develop a program to compare the number of meals prepared, served and left over across several days. Calculate wastage percentage and identify days exceeding the allowed limit.

2. Objectives

- Record meals prepared and served.
- Calculate leftover meals and wastage percentage.
- Identify days with high food wastage.

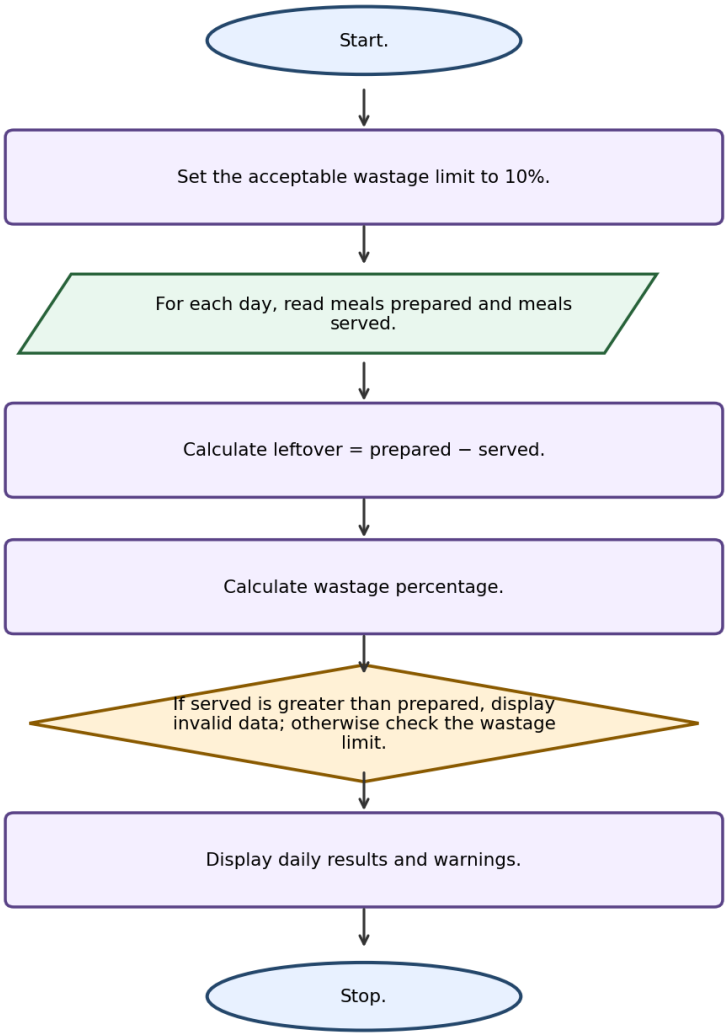
3. Assumptions

- The sample monitors three days.
- Meals prepared must be greater than zero.
- Wastage percentage = leftover meals ÷ prepared meals × 100.
- A day is flagged when wastage is greater than 10%.

4. Algorithm (Diagram / Flowchart)

The following diagram shows the main steps of the algorithm.

FLOWCHART



Algorithm steps:

81. Start.
82. Set the acceptable wastage limit to 10%.
83. For each day, read meals prepared and meals served.
84. Calculate leftover = prepared − served.
85. Calculate wastage percentage.
86. If served is greater than prepared, display invalid data; otherwise check the wastage limit.
87. Display daily results and warnings.
88. Stop.

5. Pseudocode

```
BEGIN
  limit = 10
```

```
  FOR each day
    INPUT prepared, served
    IF prepared <= 0 OR served < 0 OR served > prepared
      DISPLAY invalid data
    ELSE
      leftover = prepared - served
      wastage = leftover / prepared * 100
      DISPLAY leftover and wastage
      IF wastage > limit: DISPLAY high wastage warning
    END IF
  END FOR
END
```

6. Python Source Code

Food Waste Monitoring System.py - C:/Users/India/AppData/Local/Programs/Python/Python313/Food Waste Monitoring System.py (3.13.15)

File Edit Format Run Options Window Help

```
days = ["Monday", "Tuesday", "Wednesday"]
limit = 10
for day in days:
    print(day)
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))
    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid data")
    else:
        leftover = prepared - served
        wastage = (leftover / prepared) * 100
        print("Leftover meals:", leftover)
        print("Wastage percentage:", round(wastage, 2), "%")
        if wastage > limit:
            print("Warning: High food wastage")
        else:
            print("Wastage is within the limit")
```

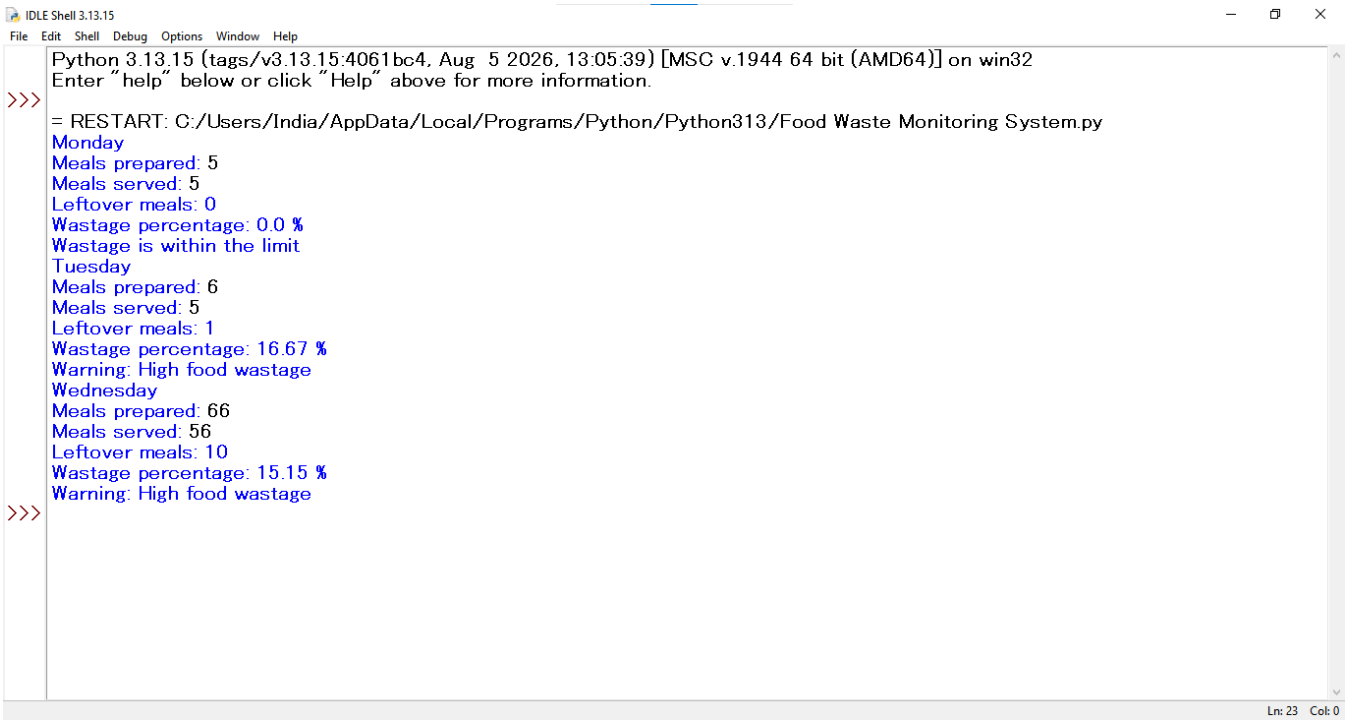
Ln: 7 Col: 0

7. Test Cases

Input / Test	Expected result
Prepared=100, served=85	Leftover=15; wastage=15%; high wastage warning
Prepared=100, served=95	Leftover=5; wastage=5%; within limit

8. Sample Output Screenshot

Illustrative terminal-style output for the sample test:



9. Conclusion

The program calculates food wastage percentages and identifies days that exceed the allowed limit using arithmetic and conditional statements.